DAY 1 - OOPS Bootcamp (Intro)

ITSRUNTYM

Day 1: Introduction to OOP, Classes, Objects, Constructors, this, Static vs Non-Static

1. What is OOP? (Object-Oriented Programming)

Definition:

OOP (Object-Oriented Programming) is a programming paradigm that organizes code using **objects** — instances of **classes**, which bundle data and behaviour together.

Core Idea:

Instead of writing code as procedures or functions (like procedural programming), you model your code around **real-world entities** — called **objects**.

(Procedural programming at the end)

Example from Real Life:

- **Car** is an object.
- It has **data** (color, speed, brand).
- It has **behaviors** (drive, brake, accelerate).

Benefits of OOP:

Benefit	Description
Modularity	Code is divided into small, manageable pieces (classes).
Reusability	Classes can be reused across programs.
Scalability	Easier to add new features.
Maintainability	Changes are easier as each class handles its responsibility.

Real-World Mapping	Makes code more intuitive and readable.

Four Main Pillars of OOP (Covered in upcoming days):

1. **Encapsulation** – Wrap data and behaviour in a class.
2. **Abstraction** – Hide complex details and expose only essentials.
3. **Inheritance** – Derive new classes from existing ones.
4. **Polymorphism** – Same action, different behaviour (like method overloading/overriding).

2. Classes and Objects

What is a Class?

- A **class** is a **blueprint** or template for creating objects.
- It defines the **properties (fields)** and **behaviors (methods)**.

What is an Object?

- An **object** is an **instance** of a class.
- It occupies memory and holds actual values.

Java Example:

```
// Define a class
class Car {
    String brand;
    int speed;

// Behavior
    void drive() {
        System.out.println(brand + " is driving at " + speed + " km/h");
    }
}

public class Main {
    public static void main(String[] args) {
        // Create Object 1
        Car car1 = new Car();
        car1.brand = "Tesla";
        car1.speed = 100;
        car1.drive();
```

```
// Create Object 2
        Car car2 = new Car();
        car2.brand = "BMW";
        car2.speed = 150;
        car2.drive();
    }
}
```

Output:

```
Tesla is driving at 100 km/h
BMW is driving at 150 km/h
```

Why Use Classes/Objects?

- Group related variables and methods together.
- Reduce complexity and improve code organization.
- Helps follow DRY (Don't Repeat Yourself) principle.

3. Constructors and this Keyword

What is a Constructor?

- A **constructor** is a special method **used to initialize** an object.
- It's **called automatically** when the object is created.
- It **must have the same name as the class**.
- It **has no return type** (not even void).

Types of Constructors:

Type	Description
Default Constructor	Provided by Java if no constructor is defined
No-Argument Constructor	A constructor with no parameters
Parameterized Constructor	Takes parameters to initialize object fields
Copy Constructor	Java does not have a built-in "copy constructor" like C++, but you can define your own copy constructor manually in Java.

Java Example: Using Parameterized Constructor and this

```
class Student {
    String name;
    int age;

// Parameterized Constructor
    Student(String name, int age) {
        this.name = name; // 'this' refers to the current object
        this.age = age;
    }
    void printDetails() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Karan", 21);
        Student s2 = new Student("Arjun", 22);
        s1.printDetails();
        s2.printDetails();
    }
}
```

Output:

```
Name: Karan, Age: 21
Name: Arjun, Age: 22
```

What is this keyword?

- Refers to the **current object**.
- Helps distinguish between **instance variables** and **parameters** if they have the same name.

Why use Constructors?

- Automatically initialize data.
- Avoid writing repetitive set methods.
- Cleaner code when creating objects with values.

What is a Copy Constructor?

Definition:

A **copy constructor** is a constructor that **creates a new object by copying the fields of an existing object**.

Syntax (you define it manually):

```
ClassName(ClassName otherObject) {
    // copy fields from otherObject
}
```

Why isn't there a built-in copy constructor in Java?

Unlike C++, Java prefers:

1. **Cloning** (via clone() and Cloneable interface)
2. **Manual copy constructors** written by developers
3. **Object serialization or factory methods** in frameworks

This gives developers **explicit control** over deep vs shallow copy.

Example: Manual Copy Constructor in Java

```
class Student {
    String name;
    int age;

// Parameterized constructor
    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

//  Copy Constructor
    Student(Student other) {
        this.name = other.name;
        this.age = other.age;
    }
void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Karan", 21);
        Student s2 = new Student(s1); //  Using copy constructor
s1.display();
        s2.display();
    }
```

```
}
```

Output:

```
Name: Karan, Age: 21
Name: Karan, Age: 21
```

Shallow vs Deep Copy

Type	What It Does
Shallow Copy	Copies object reference only
Deep Copy	Creates a **new copy** of the object and all its inner objects

For example, if a field is an object (e.g., an array or custom class), and you just assign it, both original and copied object will share the **same reference** — that's **shallow copy**.

Example of Problem with Shallow Copy:

```
class Address {
    String city;
    Address(String city) { this.city = city; }
}

class Person {
    String name;
    Address address;

Person(String name, Address address) {
        this.name = name;
        this.address = address;
    }

// Copy Constructor - SHALLOW COPY
    Person(Person other) {
        this.name = other.name;
        this.address = other.address; // shared reference!
    }
}

public class Main {
    public static void main(String[] args) {
        Address addr = new Address("Delhi");
        Person p1 = new Person("Karan", addr);
```

```
        Person p2 = new Person(p1);
p2.address.city = "Mumbai"; // modifies p1 too
System.out.println(p1.address.city); // Mumbai
        System.out.println(p2.address.city); // Mumbai
    }
}
```

Deep Copy Constructor

```
Person(Person other) {
    this.name = other.name;
    this.address = new Address(other.address.city); // deep copy
}
```

What is static Keyword in Java?

Definition:

The static keyword in Java is used **to create class-level members** — fields or methods — that belong to the **class itself** rather than any individual object.

What Happens When You Use static?

- Memory for static members is **allocated only once** when the class is loaded.
- All instances of the class **share the same static member**.
- You **don't need an object** to access static members — use ClassName.member.

4. Static vs Non-Static Members

What is static?

- **Belongs to the class**, not the instance.
- Shared among all objects.
- Can be accessed using **ClassName.memberName** or object.

What is non-static?

- **Belongs to the object**.
- Each object gets its **own copy**.

Java Example: Static vs Non-Static

```
class Counter {
    static int staticCount = 0; // shared across all objects
    int nonStaticCount = 0;     // unique per object
void increment() {
        staticCount++;
        nonStaticCount++;
        System.out.println("Static: " + staticCount + ", Non-static: " +
nonStaticCount);
    }
}
public class Main {
    public static void main(String[] args) {
        Counter c1 = new Counter();
        Counter c2 = new Counter();

c1.increment();  // Static: 1, Non-static: 1
        c2.increment();  // Static: 2, Non-static: 1
    }
}
```

Output:

```
Static: 1, Non-static: 1
Static: 2, Non-static: 1
```

When to Use static:

- For shared values: e.g., count, PI, MAX_VALUE.

Differences Between Static and Non-Static:

Feature	Static	Non-Static
Belongs to	Class	Object
Memory	Allocated once	Per object
Access	ClassName or object	Object only
Use Case	Shared data	Object-specific data

Where Can We Use static in Java?

Context	Meaning
static variable	Shared across all objects of the class
static method	Belongs to class, can be called without object
static block	Runs once when class is loaded
static class (nested only)	Inner class that doesn't require outer class object

Let's Explore Each of These in Detail

1. static Variable (also known as class variable)
Description:

- Belongs to **class**, not to objects.
- **Only one copy** exists in memory, shared across all instances.

When to Use:

- When a value should be **shared** by all objects (e.g., a counter, constants).

Example:

```
class Student {
    String name;
    static String college = "IIT"; // static variable
Student(String name) {
        this.name = name;
    }
void display() {
        System.out.println(name + " from " + college);
    }
}
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Karan");
        Student s2 = new Student("Arjun");
s1.display();
        s2.display();
    }
}
```

Output:

Karan from IIT
Arjun from IIT

Changing college affects **all objects** since it's shared.

2. static Method
Description:

- Can be called **without creating an object**.
- Can **only access static data** directly.
- Cannot use this keyword (no object context).

When to Use:

- Utility methods, helpers (like Math.max, Arrays.sort)
- Factory methods (used in patterns)

Example:

```
class MathUtil {
    static int square(int x) {
        return x * x;
    }
}
public class Main {
    public static void main(String[] args) {
        System.out.println(MathUtil.square(5)); // no need to create object
    }
}
```

Output:

```
25
```

You can also call static methods from other static methods (like main).

3. static Block (Static Initialization Block)
Description:

- A block that is executed **once**, when the class is **loaded**.
- Useful to **initialize static variables**.

When to Use:

- Complex static initialization (e.g., loading files, config, DB connections).

Example:

```
class Config {
    static String appName;
static {
        appName = "OOP Bootcamp";
        System.out.println("Static block executed");
    }
}
public class Main {
    public static void main(String[] args) {
        System.out.println("App: " + Config.appName);
    }
}
```

Output:

```
Static block executed
App: OOP Bootcamp
```

Runs **before main()** if the class is accessed.

4. static Class (Nested Static Class)
Description:

- A **static nested class** is a class defined **inside another class**, marked static.
- Does **not require an object** of the outer class to be created.

When to Use:

- Group classes logically.
- Encapsulation when outer class object is not needed.

Example:

```
class Outer {
    static class Inner {
        void show() {
            System.out.println("Inside static nested class");
        }
    }
}
public class Main {
    public static void main(String[] args) {
        Outer.Inner obj = new Outer.Inner(); // no Outer object needed
        obj.show();
    }
```

```
}
```

Output:

Inside static nested class

Summary Table

Static Type	Usage	Access	Notes
Static Variable	Shared data	ClassName.var or object	1 copy only
Static Method	Utility/function	ClassName.method()	Cannot access non-static members
Static Block	Init logic	Auto-executes on class load	Runs once
Static Nested Class	Inner class logic	Outer.Inner	Doesn't need outer object

Key Notes

- Static members are **common for all objects**.
- Static methods **can only access static data**.
- You **cannot override static methods** in true polymorphic sense.
- Static variables can also be final to create **constants**.

What is Procedural Programming?

Definition:

Procedural Programming is a programming paradigm where the code is organized into **procedures** (also called functions or routines), and the focus is on **performing operations step-by-step** in a **sequence**.

Key Characteristics:

Feature	Description
Procedure-based	Code is divided into functions/procedures.

Top-down approach	Code is executed step-by-step from top to bottom.
Global data	Functions often operate on shared global data.
Reusability	Some reusability via functions, but **less modular** than OOP.
No concept of objects	Unlike OOP, it does **not use classes or objects**.

Real-Life Analogy:
Think of a **recipe** for making tea:

1. Boil water.
2. Add tea leaves.
3. Add sugar.
4. Strain and serve.

You follow a **sequence of steps** — this is exactly how **procedural programming** works.

Procedural Programming Example in Java
Even though Java is an object-oriented language, you can still write code **procedurally** using static methods.

```
public class TeaMaker {
static void boilWater() {
        System.out.println("Boiling water...");
    }
static void addTeaLeaves() {
        System.out.println("Adding tea leaves...");
    }
static void addSugar() {
        System.out.println("Adding sugar...");
    }
static void serveTea() {
        System.out.println("Straining and serving tea...");
    }
public static void main(String[] args) {
        boilWater();
        addTeaLeaves();
        addSugar();
        serveTea();
    }
}
```

Output:

Boiling water...
Adding tea leaves...
Adding sugar...
Straining and serving tea...
This is **purely procedural** — you are not using any class instances or encapsulation. You are just organizing your logic in **functions** and calling them in sequence.

Practice Task

```
class BankAccount {
    static double interestRate;
    String accountHolder;
    double balance;
BankAccount(String name, double balance) {
        this.accountHolder = name;
        this.balance = balance;
    }
void showDetails() {
        System.out.println(accountHolder + "'s Balance: " + balance + ", Interest
Rate: " + interestRate);
    }
static void updateInterest(double newRate) {
        interestRate = newRate;
    }
}
public class Main {
    public static void main(String[] args) {
        BankAccount.updateInterest(6.5);
BankAccount acc1 = new BankAccount("Karan", 10000);
        BankAccount acc2 = new BankAccount("Neha", 15000);
acc1.showDetails();
        acc2.showDetails();
    }
}
```